

SBA301 – SLOT 18

CHAPTER 13 – CONNECTING RESTFUL WEB SERVICE TO JPA

PART B: REST CONTROLLER • CRUD • REQUEST/RESPONSE • JPA INTEGRATION + LAB 04

Tư duy trung tâm của Slot 18: Controller không chỉ “gắn annotation”. Nó là HTTP boundary: nhận request, bind/validate input, gọi Service đúng use case, chọn status/headers/body đúng contract và không để chi tiết persistence tràn ra ngoài. Luồng cần nhìn được: Client → DispatcherServlet → Controller → Service → Repository → Hibernate/JPA → SQL Server → JSON response.

Thông tin	Giá trị
Môn học	SBA301 – Integrate Single Page Application with Spring Boot
Slot	18
Chủ đề	Chapter 13 Part B – REST Controller + Spring Data JPA Integration
CLO chính	CLO4 – Describe/Apply connecting RESTful Web Service with JPA trong course sequence
Ngữ liệu xuyên suốt	demo_slot18 / Orchid Management – Lab 04
Môi trường khóa học	JDK 21 • Spring Boot 3.x track • Maven • Spring Web • Spring Data JPA • SQL Server • Postman/Swagger
Thời lượng gợi ý	135 phút trên lớp + Lab 04/tự học ngoài giờ
Learning flow	Prepare → Understand → Trace → Implement → Break → Diagnose → Verify → Transfer

Scope lock: Slot 18 hoàn tất cầu nối REST ↔ JPA. Không dạy lại sâu entity/relationship của Slot 17 và không đi sâu Security của các slot sau. Validation, exception handling, DTO và API design production được dùng như extension để sinh viên hiểu boundary chuyên nghiệp.

CÁCH SỬ DỤNG GIÁO TRÌNH

Giáo trình tiếp tục template Slot 16–17 nhưng đổi trọng tâm từ “persistence bên trong” sang “HTTP boundary + end-to-end request flow”. Lab 04 Orchid Management là bài thực hành bắt buộc; ví dụ e-commerce được dùng để transfer kiến thức sang dự án lớn.

Pha học	Sinh viên làm gì?	Bằng chứng
Prepare	Chạy baseline Slot 17/Lab 04; xác nhận DB và application start.	Baseline run + Git checkpoint
Understand	Phân biệt mapping, binding, status, body, Controller/Service/Repository boundary.	API contract map
Trace	Theo dõi startup và một request từ DispatcherServlet tới SQL Server.	Execution trace
Implement	Hoàn thiện CRUD controller và REST/JPA integration.	Source + Postman/Swagger evidence
Break	Cố ý sai mapping/status/body/relationship.	Failure evidence
Diagnose	Đọc 404/400/415/500, stack trace, SQL log.	Debug note
Verify	Kiểm tra hậu điều kiện trong API và database.	Test matrix + DB evidence
Transfer	Thiết kế resource/API cho module group project/e-commerce.	API design worksheet

AI rule: AI được dùng để khám phá execution rule, production design và technology update; sinh viên phải đối chiếu với source code, logs, Spring documentation và chạy thử. AI không được thay thế evidence.

0. SINH VIÊN CẦN CHUẨN BỊ GÌ TRƯỚC KHI HỌC BUỔI NÀY

0.1. Kiến thức cần ôn lại

- Slot 17: JPA/ORM/Hibernate/Spring Data JPA; @Entity, @Id, @GeneratedValue, @Column.
- Repository: JpaRepository, derived query; Service và transaction boundary.
- Relationship: Many-to-One / One-to-Many / Many-to-Many; owning side, mappedBy, FK/join table; JSON recursion awareness.
- Slot 12–16: HTTP methods/status codes, @RestController cơ bản, CRUD, OpenAPI, paging/sorting, testing/MockMvc.
- HTTP request gồm method + URL + headers + optional body; response gồm status + headers + optional body.

0.2. Công cụ và project baseline

Thành phần	Yêu cầu	Kiểm tra nhanh
JDK	21	java -version
Maven	3.6.3+ phù hợp Boot 3.x	mvn -v
IDE	IntelliJ IDEA	Project import/build không lỗi
Dependencies	Spring Web, Spring Data JPA, SQL Server Driver, Validation (nếu dùng), Test	Maven reload xanh
SQL Server	Database kết nối được	SSMS/SQL client + startup log
Postman/Swagger	Gửi request và xem JSON/status	GET smoke test
Git	Baseline sạch	git status / git log

0.3. Pre-class 3–2–1 Learning Note

3 điều đã biết	2 điều còn mơ hồ	1 câu hỏi muốn kiểm chứng
REST Controller nhận request; Service chứa business logic; Repository truy cập DB.	@PathVariable khác @RequestParam ở boundary nào?; ResponseEntity có giá trị gì?	Ví dụ: một GET /api/orchids/1 thực sự đi qua những method nào?

0.4. Pre-flight checklist

- Project Slot 17 compile và start trước khi sửa controller.
- SQL Server database tồn tại; application.properties/yaml đúng.
- Có ít nhất OrchidCategory và Orchid seed để test.
- Commit/tag gợi ý: s18-before-rest-controller.
- Bật SQL log ở mức phù hợp cho workshop trace.
- Postman collection hoặc Swagger UI sẵn sàng.

Baseline checkpoint

```
mvn -q -DskipTests compile
mvn test
# Sau đó chạy application từ IDE hoặc:
mvn spring-boot:run
```

I. TÓM TẮT CHỦ ĐỀ & MỤC TIÊU BÀI HỌC

1.1. Slot 17 → Slot 18: từ persistence core đến HTTP boundary

Nền tảng Slot 17	Đã làm được	Câu hỏi Slot 18 trả lời
Entity/relationship	Map Java object ↔ relational model	Client gửi/nhận representation nào?
JpaRepository	CRUD/query abstraction	Endpoint nào gọi use case nào?
Service/transaction	Business rule + transaction boundary	Controller phải mỏng tới mức nào?
SQL Server	Persist dữ liệu thật	Status/body có phản ánh hậu điều kiện DB không?
JSON recursion awareness	Biết object graph có thể gây lỗi	DTO/serialization boundary nên thiết kế ra sao?

1.2. Mục tiêu học tập

1. Giải thích vai trò của @RestController và request mapping trong Spring MVC.
2. Dùng đúng @RequestMapping/@GetMapping/@PostMapping/@PutMapping/@DeleteMapping/@PatchMapping theo HTTP semantics.
3. Phân biệt @PathVariable, @RequestParam và @RequestBody; hiểu binding/conversion/JSON deserialization.
4. Thiết kế Controller → Service → Repository đúng separation of concerns; tránh controller gọi DB trực tiếp.
5. Dùng ResponseEntity/status code phù hợp cho GET/POST/PUT/DELETE và error cases.
6. Trace được một request từ client qua Spring MVC/JPA tới SQL Server và response quay ngược lại.
7. Kiểm chứng CRUD bằng Postman/Swagger và kiểm tra hậu điều kiện trong database.
8. Nhận biết production extensions: DTO, validation, global exception handling, pagination, API versioning, auth, observability.
9. Chuyển kiến thức sang một resource trong project quy mô lớn, đặc biệt e-commerce.

1.3. Mental model của Slot 18

Hai flow phải phân biệt: startup flow và request flow

Application startup
 SpringApplication.run(...)
 -> component scan / auto-configuration
 -> DataSource + JPA/Hibernate + Repository proxy
 -> Service bean
 -> Controller bean + RequestMapping registry
 -> embedded servlet container + DispatcherServlet

Runtime request
 HTTP Client
 -> DispatcherServlet
 -> HandlerMapping
 -> OrchidController.method(...)
 -> OrchidService.method(...)
 -> OrchidRepository.method(...)
 -> Hibernate / EntityManager
 -> JDBC Driver
 -> SQL Server
 <- result / entity
 <- JSON serialization
 HTTP Response

1.4. “API chạy được” chưa đủ

Mức	Biểu hiện	Rủi ro
-----	-----------	--------

Mức	Biểu hiện	Rủi ro
Demo beginner	Endpoint trả JSON, controller gọi repository trực tiếp.	Business rule trộn, khó test, khó transaction/refactor.
Course-ready	Controller → Service → Repository; status/mapping đúng; DB verify.	Đủ mục tiêu Slot 18.
Production-ready direction	DTO, validation, error contract, auth, versioning, pagination, observability, idempotency...	Cần thêm design/governance ngoài core lab.

II. TRỌNG TÂM CỐT LỖI & KỸ NĂNG PHẢI NẮM VỮNG

2.1. @RestController – HTTP boundary của application

`@RestController` kết hợp vai trò controller và response body serialization: method return value thường được chuyển thành JSON qua HTTP message converters. Trong kiến trúc 3-layer, Controller nên chịu trách nhiệm HTTP concerns; business logic thuộc Service; persistence concerns thuộc Repository.

Layer	Nên làm	Không nên làm
Controller	Routing, binding, validation trigger, status/headers/body, gọi use case	Query DB trực tiếp; nhồi business rule; transaction phức tạp
Service	Business rule, orchestration, transaction boundary	Biết URL/HTTP status chi tiết
Repository	CRUD/query persistence abstraction	Xử lý HTTP hoặc business workflow

2.2. Request mapping – URL + HTTP method là contract

Annotation	HTTP semantics	Ví dụ
@RequestMapping	Base path/criteria chung	@RequestMapping("/api/orchids")
@GetMapping	Read representation	GET /api/orchids/{id}
@PostMapping	Create new resource / command	POST /api/orchids
@PutMapping	Full replacement/update theo contract	PUT /api/orchids/{id}
@PatchMapping	Partial update	PATCH /api/orchids/{id}
@DeleteMapping	Delete resource	DELETE /api/orchids/{id}

Rule: Không thiết kế endpoint bằng động từ kiểu /getAllOrchids, /createOrchid, /deleteOrchid nếu resource-oriented REST đã đủ biểu đạt bằng method + resource URL.

2.3. @PathVariable vs @RequestParam vs @RequestBody

Cơ chế	Dùng khi	Ví dụ	Sai lầm thường gặp
@PathVariable	Identity/sub-resource nằm trong path	/orchids/{id}	Đưa filter tùy chọn thành path dài khó mở rộng
@RequestParam	Filter/search/page/sort/options	?name=cat&page=0&size=20	Dùng query param cho resource identity chính
@RequestBody	Representation/command gửi trong body	JSON create/update	Đọc body bằng String rồi parse thủ công

Binding examples

```

@GetMapping("/{id}")
public ResponseEntity<Orchid> getById(@PathVariable Long id) { ... }

@GetMapping
public List<Orchid> search(@RequestParam(required = false) String name) { ... }

@PostMapping
public ResponseEntity<Orchid> create(@RequestBody Orchid orchid) { ... }

```

2.4. ResponseEntity – status + headers + body

Spring Framework mô tả ResponseEntity như response body có thêm khả năng điều khiển status và headers. Ở Slot 18, sinh viên dùng nó để biểu đạt API contract rõ ràng thay vì trả mọi trường hợp về 200.

Scenario	Status gợi ý	Body
GET list/detail success	200 OK	resource/list
POST create	201 Created	created resource; production có thể thêm Location
PUT update	200 OK hoặc 204 theo contract	updated resource hoặc empty
DELETE success	204 No Content	empty
Invalid input	400 Bad Request	error contract
Not found	404 Not Found	error contract
Unsupported media type	415 Unsupported Media Type	error contract

2.5. Request body → Java object → JPA entity

Serialization/deserialization flow

```

JSON request
-> HttpMessageConverter / Jackson
-> Java object (Entity hoặc DTO)
-> Controller
-> Service
-> Repository.save(...)
-> Hibernate persistence context
-> SQL INSERT/UPDATE
-> SQL Server

```

Boundary warning: Lab có thể nhận/trả Entity để tập trung vào REST↔JPA. Trong dự án lớn, không mặc định expose Entity trực tiếp vì coupling schema, lazy associations, sensitive fields và JSON recursion.

2.6. Constructor injection – hiểu Dependency Injection bằng runtime graph

Khuyến nghị cho code mới: constructor injection

```

@RestController
@RequestMapping("/api/orchids")
public class OrchidController {
    private final OrchidService service;

    public OrchidController(OrchidService service) {
        this.service = service;
    }
}

```

Slide Chapter 13 minh họa khái niệm dependency injection bằng @Autowired. Với code mới, constructor injection giúp dependency rõ ràng, dễ test và hỗ trợ immutable field. Trọng tâm là hiểu Spring tạo/inject bean, không học thuộc một annotation duy nhất.

2.7. Controller → Service → Repository: method chain phải trace được

Method-chain mental model

```

GET /api/orchids/5
OrchidController.getById(5)
-> OrchidService.getById(5)
-> OrchidRepository.findById(5)
-> SimpleJpaRepository.findById(...)
-> EntityManager.find(...)
-> Hibernate SQL SELECT ... WHERE orchid_id=?
-> SQL Server
<- Orchid / Optional<Orchid>
<- ResponseEntity<...>
<- JSON

```

2.8. Relationship + JSON: database đúng nhưng API vẫn có thể sai

Với Category 1–N Orchid, JPA mapping đúng không đồng nghĩa JSON response an toàn. Bidirectional object graph có thể serialize lặp vô hạn hoặc tải quá nhiều dữ liệu. Slot 17 đã cảnh báo JSON recursion; Slot 18 biến cảnh báo này thành API boundary decision.

Giải pháp	Ưu điểm	Trade-off / dùng khi
DTO/Response model	Tách API contract khỏi entity; kiểm soát fields	Khuyến nghị production; cần mapping
@JsonIgnore	Nhanh cho demo	Couple serialization vào entity; có thể che mất data
Managed/Back reference annotations	Xử lý một số bidirectional graphs	Jackson-specific; dễ phức tạp
Projection/query DTO	Giảm data fetch/serialization	Thiết kế query/contract rõ hơn

2.9. Error handling – từ exception Java thành error contract HTTP

Production extension – global exception handling

```

@RestControllerAdvice
public class ApiExceptionHandler {
    @ExceptionHandler(ResourceNotFoundException.class)
    ResponseEntity<ApiError> handleNotFound(ResourceNotFoundException ex) {
        return ResponseEntity.status(HttpStatus.NOT_FOUND)
            .body(new ApiError("NOT_FOUND", ex.getMessage()));
    }
}

```

Chapter 13 liệt kê Error Handling là additional consideration. Core Lab 04 có thể xử lý đơn giản; dự án lớn nên nhất quán error code/message/details/timestamp/traceId theo contract thay vì mỗi controller tự trả lỗi khác nhau.

2.10. Validation – đừng để invalid request đi tới database

Production extension – Bean Validation

```
public record OrchidRequest(
    @NotBlank String orchidName,
    @NotNull Long categoryId,
    String orchidDescription,
    Boolean natural
) {}

@PostMapping
public ResponseEntity<?> create(@Valid @RequestBody OrchidRequest request) { ... }
```

2.11. CRUD semantics và hậu điều kiện

Operation	Pre-condition	Action	Post-condition cần verify
POST	Body hợp lệ; category tồn tại	service.create	Record mới có ID; GET/DB thấy record
GET	ID/filter hợp lệ	service.read/query	Không mutate DB; representation đúng
PUT	ID tồn tại; body hợp lệ	service.update	Fields theo contract được cập nhật
DELETE	ID tồn tại	service.delete	GET sau delete → 404; row bị xóa/soft-delete theo rule

2.12. Mapping table – request nào tới method nào?

HTTP Request	Controller method	Service use case	Repository
GET /api/orchids	getAll()	findAll()	findAll()
GET /api/orchids/{id}	getById(id)	getById(id)	findById(id)
GET /api/orchids?name=x	search(name)	searchByName(name)	findByOrchidNameContainingIgnoreCase(...)
POST /api/orchids	create(body)	create(...)	save(...)
PUT /api/orchids/{id}	update(id, body)	update(...)	findById + save
DELETE /api/orchids/{id}	delete(id)	delete(...)	exists/find + delete

III. ỨNG DỤNG THỰC TIỄN KIẾN THỨC TRONG SẢN PHẨM

3.1. Case study xuyên suốt – Orchid Management

Resource	Fields chính	Relationship	API concern
OrchidCategory	id, name	1 Category – N Orchids	Create/read categories; reference integrity
Orchid	id, name, description, natural, attractive, url	N Orchids – 1 Category	CRUD, search, representation, status/error

3.2. Từ use case đến API contract

Use case	Endpoint	Input	Output/Status
Browse orchids	GET /api/orchids	query params optional	200 + list/page
View detail	GET /api/orchids/{id}	path id	200 or 404
Create orchid	POST /api/orchids	JSON body	201 + created
Update orchid	PUT /api/orchids/{id}	id + JSON	200/404/400
Delete orchid	DELETE /api/orchids/{id}	id	204/404
Search by name/category	GET /api/orchids?name=&categoryId=	query params	200 + filtered list/page

3.3. Transfer sang e-commerce quy mô lớn

Một hệ thống thương mại điện tử không chỉ có “Product CRUD”. Resource model thường gồm Catalog/Product, Category, Inventory, Price, Cart, Order, Payment, Shipment, Customer/Identity, Promotion. Mỗi bounded area có contract, rule và lifecycle khác nhau; Controller → Service → Repository vẫn là mental model nền nhưng không nên biến tất cả thành một controller/service khổng lồ.

Resource/Module	Endpoint mẫu	Business concern	Scale concern
Catalog	GET /api/v1/products	filter/search/category	pagination, cache, search engine
Inventory	GET/POST /api/v1/inventory-reservations	reserve/release stock	concurrency, idempotency
Cart	GET/POST /api/v1/carts/{id}/items	cart lifecycle	session/user ownership
Order	POST /api/v1/orders	checkout snapshot	transaction/saga, status machine
Payment	POST /api/v1/payments	authorize/capture	idempotency, external gateway
Shipment	GET /api/v1/orders/{id}/shipments	tracking	event-driven integration

3.4. Beginner design vs production design

Vấn đề	Người mới thường làm	Production direction
URL	/getProducts, /createOrder	Resource-oriented URL + HTTP method
Model	Expose Entity trực tiếp	DTO/request/response model
Error	200 + message string	Status đúng + standardized error contract
Versioning	Không có strategy	Version only when needed; migration/deprecation policy
Auth	Check role thủ công trong controller	Spring Security/policy layer; 401/403 semantics
Large list	findAll()	Pagination/filter/sort; bounded page size
Duplicate POST	Create lại	Idempotency key/unique business key khi cần
Observability	System.out.println	Structured logs, metrics, traces, correlation/trace id
Integration	Controller gọi external service trực tiếp	Application/service boundary + resilient HTTP client/event
Database	One transaction for everything	Transaction boundary per use case; distributed consistency strategy

IV. HƯỚNG DẪN CHI TIẾT THỰC HIỆN VÀ TRIỂN KHAI

A. Demo baseline – cấu trúc project

Project tree

```
demo_slot18/
├── src/main/java/...
│   ├── DemoSlot18Application.java
│   ├── controller/
│   │   └── OrchidController.java
│   ├── service/
│   │   ├── OrchidService.java
│   │   └── OrchidServiceImpl.java
│   ├── repository/
│   │   ├── OrchidRepository.java
│   │   └── OrchidCategoryRepository.java
│   ├── entity/
│   │   ├── Orchid.java
│   │   └── OrchidCategory.java
│   ├── dto/           # extension nếu áp dụng
│   └── exception/     # extension nếu áp dụng
└── src/main/resources/application.properties
```

B. Bước 1 – Main class và startup trace

Entry point

```
@SpringBootApplication
public class DemoSlot18Application {
    public static void main(String[] args) {
        SpringApplication.run(DemoSlot18Application.class, args);
    }
}
```

10. JVM gọi main().
11. SpringApplication.run tạo/refresh ApplicationContext.
12. Component scanning phát hiện @Controller/@Service/@Repository/@Entity-related configuration.
13. Auto-configuration thiết lập MVC, DataSource, JPA/Hibernate theo classpath/properties.
14. Spring Data tạo repository proxy.
15. Spring tạo Service và Controller bean theo dependency graph.
16. Embedded servlet container start; DispatcherServlet và mappings sẵn sàng nhận HTTP.

Trace evidence: Sinh viên mở startup log và tìm port, DataSource/JPA/Hibernate initialization, repository scanning và “Started ...Application”. Không đoán bean lifecycle chỉ từ tên annotation.

C. Bước 2 – Entity/relationship readiness

Orchid – relationship side

```

@Entity
public class Orchid {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long orchidId;

    private String orchidName;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "category_id", nullable = false)
    private OrchidCategory category;
}

```

Phần mapping chi tiết thuộc Slot 17. Slot 18 chỉ verify rằng API create/update đọc category đúng và database lưu FK đúng.

D. Bước 3 – Repository contract

Repository

```

public interface OrchidRepository extends JpaRepository<Orchid, Long> {
    List<Orchid> findByOrchidNameContainingIgnoreCase(String name);
}

```

E. Bước 4 – Service use cases và transaction boundary

Service skeleton

```

@Service
public class OrchidServiceImpl implements OrchidService {
    private final OrchidRepository orchidRepository;
    private final OrchidCategoryRepository categoryRepository;

    public OrchidServiceImpl(OrchidRepository orchidRepository,
        OrchidCategoryRepository categoryRepository) {
        this.orchidRepository = orchidRepository;
        this.categoryRepository = categoryRepository;
    }

    @Transactional(readOnly = true)
    public List<Orchid> findAll() {
        return orchidRepository.findAll();
    }

    @Transactional
    public Orchid create(Orchid orchid) {
        return orchidRepository.save(orchid);
    }
}

```

F. Bước 5 – REST Controller baseline

GET list

```

@RestController
@RequestMapping("/api/orchids")
public class OrchidController {
    private final OrchidService service;

    public OrchidController(OrchidService service) {
        this.service = service;
    }

    @GetMapping
    public ResponseEntity<List<Orchid>> getAll() {
        return ResponseEntity.ok(service.findAll());
    }
}

```

G. Bước 6 – GET detail + 404 contract

GET by id – core-friendly version

```

@GetMapping("/{id}")
public ResponseEntity<Orchid> getById(@PathVariable Long id) {
    return service.findById(id)
        .map(ResponseEntity::ok)
        .orElseGet(() -> ResponseEntity.notFound().build());
}

```

H. Bước 7 – POST + 201 Created

POST create – Lab level

```

@PostMapping
public ResponseEntity<Orchid> create(@RequestBody Orchid orchid) {
    Orchid created = service.create(orchid);
    return ResponseEntity.status(HttpStatus.CREATED).body(created);
}

```

Production extension: Có thể trả Location header bằng URI của resource mới, và nhận OrchidRequest DTO thay vì Entity.

I. Bước 8 – PUT + DELETE

Update/Delete

```

@PutMapping("/{id}")
public ResponseEntity<Orchid> update(@PathVariable Long id,
    @RequestBody Orchid input) {
    return ResponseEntity.ok(service.update(id, input));
}

@DeleteMapping("/{id}")
public ResponseEntity<Void> delete(@PathVariable Long id) {
    service.delete(id);
    return ResponseEntity.noContent().build();
}

```

J. Bước 9 – Search/filter bằng @RequestParam

@RequestParam example

```
@GetMapping("/search")
public ResponseEntity<List<Orchid>> search(
    @RequestParam(defaultValue = "") String name) {
    return ResponseEntity.ok(service.searchByName(name));
}
```

K. Bước 10 – Trace một POST end-to-end

Execution trace – conceptual framework + application methods

```
POST /api/orchids + JSON
-> DispatcherServlet.doDispatch(...)
-> RequestMappingHandlerMapping finds OrchidController.create(...)
-> Jackson converts JSON -> Orchid/DTO
-> OrchidController.create(...)
-> OrchidServiceImpl.create(...)
-> OrchidRepository.save(...)
-> SimpleJpaRepository.save(...)
-> EntityManager.persist/merge(...)
-> Hibernate generates SQL INSERT
-> JDBC Driver executes against SQL Server
<- generated ID / persisted entity
<- Controller builds 201 response
<- Jackson serializes response object -> JSON
```

Verification gate: Framework internal method names có thể thay đổi theo version. Sinh viên phải phân biệt “conceptual flow” và “method thực tế quan sát từ debugger/stack trace/source”.

L. Bước 11 – Postman/Swagger + database verification

Test	Request	Expected API	DB verify
T01	GET /api/orchids	200 + list	Không thay đổi row
T02	GET /api/orchids/1	200 + object	ID/FK khớp
T03	GET missing id	404	Không thay đổi
T04	POST valid	201	Có row mới + category_id đúng
T05	POST invalid/missing category	400/contract	Không tạo row
T06	PUT existing	200/contract	Fields thay đổi
T07	DELETE existing	204	Row bị xóa hoặc trạng thái theo rule
T08	GET deleted	404	Hậu điều kiện đúng

M. Bước 12 – Break-It / Diagnose workshop

Break	Triệu chứng dự kiến	Cách chẩn đoán
Sai @RequestMapping path	404	Check route, startup mapping, Postman URL
Bỏ @RequestBody	Body binding sai / 400	Inspect controller signature + request Content-Type
Gửi text/plain	415	Check Content-Type

Break	Triệu chứng dự kiến	Cách chẩn đoán
category_id không tồn tại	DB/FK error hoặc business error	Service rule + SQL exception
Controller gọi repository trực tiếp	Vẫn chạy nhưng boundary xấu	Architecture review, testability
Bidirectional relationship serialize vòng lặp	500/StackOverflow/huge JSON	Inspect object graph/Jackson
Transaction thiếu khi multi-step update	Partial state/rule inconsistency	Service transaction boundary
Trả 200 cho missing resource	Contract sai	Negative test / API review

N. Bước 13 – Bridge từ Slot 16: automated testing

Slot 16 đã xây testing mental model. Với JPA thật ở Slot 18, map test boundary chính xác hơn:

Boundary	Tool gợi ý	Mục tiêu
Controller/MVC	@WebMvcTest + MockMvc	mapping, binding, JSON, status, exception contract
Service	JUnit + Mockito	business rule/orchestration
Repository/JPA	@DataJpaTest	entity mapping/query
Full integration	@SpringBootTest + test DB/Testcontainers khi phù hợp	wiring + multi-layer flow
Manual smoke	Postman/Swagger	baseline/exploration, không thay regression suite

O. Lab 04 Integration Gate

Gate	Bảng chứng bắt buộc
G1 Build/Start	Application start; DB connected; no mapping error
G2 Controller	GET/POST/PUT/DELETE mapping đúng
G3 Service	Controller không chứa business/persistence logic chính
G4 JPA	Repository + relationship hoạt động; FK verify
G5 HTTP contract	Status/body hợp lý; có negative case
G6 Persistence	POST/PUT/DELETE verify hậu điều kiện trong SQL Server
G7 Trace	Giải thích method chain của ít nhất 1 request
G8 Docs	Postman collection/Swagger/README
G9 Break/Fix	Ít nhất 1 failure được chẩn đoán
G10 Explain	Human verification pass

V. KINH NGHIỆM THỰC TIỄN, LỖI THƯỜNG GẶP VÀ DEBUGGING

Vấn đề	Triệu chứng	Cách xử lý
404	URL/mapping mismatch	Check base path, method mapping, port, context path
400	Binding/validation failure	Inspect JSON shape, field types, @RequestBody/@Valid
405	Method không được phép	Check GET/POST/PUT/DELETE mapping
415	Content-Type sai	Send application/json; inspect consumes

Vấn đề	Triệu chứng	Cách xử lý
500	Unhandled exception	Read root cause + stack trace; DB/Jackson/common handler
LazyInitializationException	Access lazy association ngoài persistence context	DTO/query design; transaction boundary; không chứa bằng EAGER toàn bộ
Infinite JSON recursion	Bidirectional graph serialized	DTO or deliberate Jackson annotations
N+1 queries	List endpoint phát nhiều SQL	Query/fetch plan/projection; measure before optimizing
Business logic in controller	Controller dài/khó test	Move use case to Service
Repository leak	Controller biết query/persistence chi tiết	Keep repository behind service
200 everywhere	Client không phân biệt outcome	Use proper HTTP status/error contract
Huge findAll	Latency/memory tăng	Pagination + filters + max page size

5.1. Debugging decision tree

Diagnose from boundary inward

Request fails

- |- 404? -> URL/mapping/path variable/port
- |- 405? -> HTTP method mismatch
- |- 400? -> JSON/binding/type/validation
- |- 415? -> Content-Type/consumes
- |- 500?
 - |- stack trace points Controller/Service? -> application rule
 - |- repository/Hibernate/JDBC? -> mapping/query/DB constraint
 - |- Jackson? -> response graph/serialization

Request succeeds but data wrong

- > inspect service rule -> repository query -> generated SQL -> DB state

5.2. Best practices ở mức Slot 18

- Controller mỏng: map HTTP ↔ application use case.
- Constructor injection; dependencies explicit.
- Status code là một phần của contract, không chỉ “trang trí”.
- Mỗi destructive operation phải verify hậu điều kiện.
- Không expose relationship graph vô kiểm soát.
- Không dùng EAGER để chữa mọi LazyInitializationException.
- Không dùng findAll() cho collection production lớn.
- Error response phải nhất quán và không rò rỉ stack trace/sensitive information.
- Một bug production quan trọng nên tạo regression test khi có thể.

VI. BÀI TẬP CHỦ ĐỀ VÀ BÀI TẬP TỔNG HỢP

Bài	Yêu cầu	Evidence
1. Mapping map	Lập bảng endpoint ↔ controller ↔ service ↔ repository	Worksheet
2. GET detail	200 existing + 404 missing	Postman/MockMvc
3. POST	201 + verify SQL row/FK	API + DB screenshot/log
4. PUT	Update và verify hậu điều kiện	Before/after
5. DELETE	204 + GET lại 404	Evidence pair
6. Search	@RequestParam + derived query	Request + SQL/query evidence

Bài	Yêu cầu	Evidence
7. Relationship	Category 1–N Orchid không recursion	JSON evidence
8. Break-It	Tạo 1 lỗi 400/404/415/500 và sửa	Failure → fix note
9. Trace	Vẽ method chain một request	Execution trace
10. Production transfer	Thiết kế API cho Product/Order của e-commerce	API design sheet

6.1. Bài tập tổng hợp – Complete demo_slot18 + Lab 04

17. Hoàn thiện CRUD Orchid với SQL Server.
18. Duy trì Category–Orchid relationship và kiểm chứng foreign key.
19. Viết API contract cho ít nhất GET list/detail, POST, PUT, DELETE, search.
20. Chạy test matrix positive + negative.
21. Trace startup flow và một request flow.
22. Thực hiện ít nhất một Break-It → Diagnose → Fix.
23. Tạo README mô tả cách chạy, endpoints, DB, known limitations.
24. Áp dụng AI Self-Discovery Prompt 1 và ghi AI Verification Worksheet.

VII. FOUNDATION GLOSSARY

Thuật ngữ	Định nghĩa ngắn	Ví dụ Slot 18
@RestController	Controller có response body serialization	OrchidController
DispatcherServlet	Front controller của Spring MVC	nhận request trước controller
HandlerMapping	Tìm handler phù hợp request	GET /api/orchids/{id}
@RequestMapping	Mapping chung/criteria	/api/orchids
@GetMapping	GET shortcut	list/detail
@PostMapping	POST shortcut	create
@PutMapping	PUT shortcut	update
@DeleteMapping	DELETE shortcut	delete
@PatchMapping	PATCH shortcut	partial update extension
@PathVariable	Bind giá trị trong URL path	id
@RequestParam	Bind query parameter	name/page/size
@RequestBody	Bind request body	JSON → Java
ResponseEntity	Body + status + headers	201 Created
HttpMessageConverter	Convert HTTP body ↔ Java object	Jackson JSON converter
DTO	API-facing data shape	OrchidRequest/Response
Validation	Check input constraints	@Valid + @NotBlank
@RestControllerAdvice	Global REST exception handling	404/400 contract
Idempotency	Lặp request không gây effect ngoài dự kiến	PUT; idempotency key cho payment
API Versioning	Quản lý contract evolution	/v1 or header/version config
Observability	Logs/metrics/traces để hiểu runtime	traceld, latency, error rate

VIII. RESEARCH BASE VÀ AI-BASED LEARNING

8.1. Nguyên tắc dùng AI trong Slot 18

- AI phải đọc đúng source/project được cung cấp; nếu thiếu file/class thì phải nói “chưa đủ bằng chứng”, không tự bịa method.
- Mọi claim về execution flow phải được kiểm bằng debugger, log, stack trace hoặc code navigation.
- Mọi claim về framework/version phải kiểm bằng Spring official documentation/release notes và ghi ngày truy cập.
- Phân biệt Course Baseline (Spring Boot 3.x + JDK 21) với Current Ecosystem/Production Update.
- Output của AI là hypothesis/learning aid; evidence cuối cùng thuộc về sinh viên.

8.2. AI SELF-DISCOVERY – 3 PROMPT CHUẨN BẮT BUỘC

Mục đích: Ba prompt này tạo ba “ống kính”: (1) khám phá quy luật thực thi code, (2) chuyển kiến thức sang kiến trúc dự án lớn, (3) cập nhật xu hướng/thực tiễn công nghệ có kiểm chứng. Sinh viên lưu prompt + response + evidence vào AI Verification Log.

PROMPT 1 – Khám phá quy luật thực thi chương trình step-by-step

Copy-ready prompt 1

Tôi là người mới học Spring Boot và muốn hiểu quy luật thực thi của chương trình thay vì chỉ học thuộc annotation. Hãy đóng vai trò Senior Spring Boot Mentor và phân tích source code của project `demo\_slot18` mà tôi cung cấp.

Yêu cầu bắt buộc:

- 1) Không được tự tạo class/method không tồn tại trong source. Nếu thiếu file hoặc không đủ bằng chứng, hãy chỉ rõ cần thêm file/log nào.
- 2) Phân tích hai workflow riêng biệt:
 - A. STARTUP FLOW: bắt đầu từ `main()` / `SpringApplication.run(...)` cho đến khi application sẵn sàng nhận request.
 - B. REQUEST FLOW: chọn một API cụ thể trong project (ưu tiên GET hoặc POST Orchid) và trace từ HTTP request đến SQL Server rồi response quay về client.
- 3) Với REQUEST FLOW, hiển thị chuỗi thực thi ở dạng:


```
method1() -> method2() -> method3() -> ...
```

 và tách rõ method thuộc code của tôi với method/framework concept của Spring/Hibernate.
- 4) Với mỗi bước, ghi: file/class, method, input, output, dependency được gọi, object/bean nào đang tham gia và mục đích của bước đó.
- 5) Giải thích chi tiết các dòng lệnh/annotation quan trọng trong `DemoSlot18Application`, Controller, Service, Repository và Entity có liên quan; không cần giải thích mọi dòng thư viện nội bộ không xuất hiện trong source.
- 6) Giải thích Spring tạo và inject Controller -> Service -> Repository như thế nào; repository proxy được tạo khi nào; JPA/Hibernate chuyển `save/findById` thành SQL ở đâu về mặt khái niệm.
- 7) Với API đã chọn, tạo thêm:
 - sequence flow dạng text;
 - bảng Request -> Controller -> Service -> Repository -> Hibernate/JDBC -> SQL Server -> Response;
 - danh sách breakpoint/log point tôi nên đặt để tự kiểm chứng.
- 8) Chỉ ra 5 hiểu sai thường gặp của người mới về execution flow và đối chiếu với project này.
- 9) Cuối cùng tạo 10 câu hỏi tự kiểm tra. Mỗi câu phải buộc tôi nhìn vào source/log để trả lời, không chỉ nhớ lý thuyết.

Mục tiêu của tôi: sau khi đọc, tôi phải có thể tự mở debugger và chứng minh workflow thực thi của `demo\_slot18`.

PROMPT 2 – Chuyển kiến thức Slot 18 sang API Design trong dự án production lớn

Copy-ready prompt 2

Tiếp tục từ kiến thức Spring Boot RESTful Web Services + JPA mà tôi vừa học ở `demo\_slot18`, hãy đóng vai trò Senior Backend Engineer/Software Architect và hướng dẫn tôi cách thiết kế API theo thông lệ chuyên nghiệp trong một hệ thống thương mại điện tử quy mô lớn.

Hãy đi từ kiến thức tôi đã biết (Controller -> Service -> Repository -> Database) đến thiết kế production, không nhảy thẳng sang microservices mà không giải thích lý do.

Yêu cầu:

- 1) Chọn một e-commerce cụ thể gồm tối thiểu: Product/Catalog, Category, Inventory, Cart, Order, Payment, Shipment, Customer.
- 2) Với từng resource, đề xuất URL, HTTP method, request/response DTO, status codes, validation và error cases.
- 3) Giải thích khi nào dùng path variable, query parameter, request body; khi nào dùng POST/PUT/PATCH/DELETE.
- 4) Thiết kế pagination/filter/sort/search cho Product; tránh `findAll()` trên dữ liệu lớn.
- 5) Phân tích Controller, Service, Repository boundary và transaction boundary cho ít nhất 3 workflow: browse product, add-to-cart, checkout/create-order.
- 6) Giải thích DTO vs Entity, global exception handling, standardized error response, OpenAPI, API versioning, authentication/authorization, idempotency, optimistic locking/concurrency, caching, observability và audit logging ở mức người mới có thể hiểu.
- 7) Với checkout, vẽ workflow method/service level và chỉ ra phần nào có thể nằm trong một DB transaction, phần nào liên quan external payment/shipping và không nên gán định là một transaction đơn giản.
- 8) So sánh “bài lab nhỏ” với “production system”: code nào có thể giữ nguyên mental model, code nào cần đổi kiến trúc.
- 9) Đưa ra một cây quyết định: khi nào monolith modular đủ, khi nào mới cân nhắc tách service.
- 10) Kết thúc bằng một checklist review API trước khi merge pull request và 10 anti-patterns cần tránh.

Mọi khuyến nghị phải giải thích trade-off, không chỉ liệt kê best practices.

PROMPT 3 – Production trend & technology update có kiểm chứng

Copy-ready prompt 3

Tiếp tục phân tích các xu hướng và thông lệ chuyên nghiệp trong thiết kế API cho Spring Boot RESTful Web Services. Tôi đang học trên baseline JDK 21 + Spring Boot 3.x, vì vậy hãy phân biệt rõ: (A) kiến thức nền vững chắc và nên giữ trong course, (B) thay đổi của hệ sinh thái hiện tại, (C) nội dung chỉ nên biết để chuẩn bị migration/production chứ chưa cần áp dụng vào lab.

Hãy dùng một hệ thống thương mại điện tử thực tế làm ví dụ và thực hiện các yêu cầu sau:

- 1) Phân tích cách thiết kế resource, endpoint, URL, HTTP methods, API versioning, authentication/authorization và error contract.
- 2) Giải thích cách các API liên kết với nhau và workflow Controller -> Service -> Repository -> Database; với external service hãy chỉ rõ HTTP client/event boundary.
- 3) So sánh cách người mới thường thiết kế với cách production system lớn thường thiết kế, kèm lý do và trade-off.
- 4) Kiểm tra các thay đổi công nghệ mới nhất từ nguồn CHÍNH THỨC (Spring documentation/release notes). Ghi rõ version, ngày tài liệu/release nếu có, URL nguồn và ngày bạn truy cập. Không dùng blog thứ cấp làm nguồn duy nhất.
- 5) Tạo bảng “Course Baseline vs Current Production Radar” cho: Spring Boot, Spring Framework, Java/Jakarta, API versioning, REST clients, Jackson/JJSON, validation, testing, observability và security.
- 6) Chỉ ra API/annotation/tool nào đã deprecated hoặc thay đổi khuyến nghị; nêu migration direction nhưng không ép project lab phải nâng version giữa kỳ.
- 7) Với mỗi xu hướng, phân loại: MUST KNOW NOW / SHOULD EXPLORE / WATCH LATER.
- 8) Nếu đề cập một feature mới (ví dụ API versioning framework support, HTTP Service Clients, RestClient, observability...), hãy đưa ví dụ tối thiểu và giải thích nó giải quyết vấn đề gì so với cách cũ.
- 9) Đề xuất lộ trình 4 tuần để tôi tự cập nhật từ `demo\_slot18` lên tư duy production mà vẫn giữ project chạy ổn định.
- 10) Cuối cùng tạo “Verification Checklist”: claim nào tôi phải tự kiểm bằng official docs, compile/run, debugger, Postman/test và log.

Quy tắc quan trọng: nếu không xác minh được version/current recommendation, hãy nói chưa xác minh thay vì đoán.

8.3. AI Verification Worksheet

Prompt/claim AI	Nguồn kiểm chứng	Thử nghiệm	Kết luận của SV
Execution chain của GET/POST	Source + debugger + stack trace	Đặt breakpoint Controller/Service/Repository	
AI nói controller gọi repository trực tiếp là “đơn giản hơn”	Architecture rule + testability review	Refactor qua Service, so test/boundary	

Prompt/claim AI	Nguồn kiểm chứng	Thử nghiệm	Kết luận của SV
AI đề xuất status code	HTTP/Spring docs + API contract	Postman/MockMvc negative cases	
AI nói một feature/version mới	Spring official docs/release notes	Check dependency/version + compile sample	
AI đề xuất DTO/validation/error handler	Project source + contract	Run positive/negative tests	

8.4. Prompt bank mở rộng – nghiên cứu có định hướng

- Giải thích DispatcherServlet, HandlerMapping, HandlerAdapter và HttpMessageConverter tham gia request flow như thế nào.
- So sánh Entity trực tiếp vs DTO vs projection cho REST response có relationship.
- Phân tích N+1 problem từ một GET list endpoint và cách đo trước khi tối ưu.
- Thiết kế standardized error contract tương thích frontend React.
- So sánh path-based, header-based và media-type API versioning.
- Phân tích idempotency cho create-order/payment APIs.
- Thiết kế pagination cursor vs offset cho catalog lớn.
- Phân tích optimistic locking cho inventory/order update.
- Thiết kế observability cho request đi qua nhiều module/service.
- Đánh giá một controller hiện có và đề xuất refactor theo boundary.

IX. SINH VIÊN CẦN TỰ KIỂM TRA GÌ SAU BUỔI HỌC

9.1. Knowledge self-check

25. @RestController khác @Controller + @ResponseBody về mental model như thế nào?
26. @PathVariable và @RequestParam khác nhau ở API contract?
27. @RequestBody được chuyển từ JSON thành object bằng cơ chế nào?
28. Tại sao Controller không nên gọi JpaRepository trực tiếp trong project lớn?
29. ResponseEntity giúp biểu đạt điều gì ngoài body?
30. POST create nên trả status nào? DELETE success thường trả gì?
31. Một GET /api/orchids/1 đi qua các layer nào trước khi chạm SQL Server?
32. Tại sao relationship JPA đúng vẫn có thể làm JSON response lỗi?
33. DTO giúp giảm coupling Entity/API như thế nào?
34. 400, 404, 405, 415 và 500 gợi ý lớp lỗi nào?
35. Transaction boundary thường nên nằm ở đâu?
36. Manual Postman test khác automated regression test ở mục tiêu nào?
37. API versioning giải quyết vấn đề gì và không nên dùng để che thiết kế kém như thế nào?
38. Idempotency quan trọng đặc biệt với operation nào trong e-commerce?
39. Observability khác logging đơn thuần như thế nào?

9.2. Skill checklist

Nội dung	Đạt	Chưa đạt	Làm lại
Chạy được demo_slot18 + SQL Server	☐	☐	☐
Trace startup flow	☐	☐	☐
Trace GET/POST request flow	☐	☐	☐
Viết @RestController + mappings	☐	☐	☐
Dùng đúng PathVariable/RequestParam/RequestBody	☐	☐	☐
Trả status bằng ResponseEntity	☐	☐	☐
Verify POST/PUT/DELETE trong DB	☐	☐	☐

Nội dung	Đạt	Chưa đạt	Làm lại
Xử lý 404/400 cơ bản	☐	☐	☐
Giải thích JSON relationship risk	☐	☐	☐
Thực hiện Break-It + diagnose	☐	☐	☐
Dùng Prompt 1 và tự kiểm chứng bằng debugger/log	☐	☐	☐
Thiết kế API resource cho project lớn	☐	☐	☐

9.3. Ba mức tự đánh giá

Mức	Biểu hiện
Mức 1 – Chưa đạt	Copy controller và gọi được Postman nhưng không giải thích execution flow/status/boundary.
Mức 2 – Đạt cơ bản	Tự viết CRUD controller, trace layer flow, verify DB và xử lý negative case.
Mức 3 – Tốt	Giải thích runtime method chain, nhận diện boundary/serialization/transaction issues và transfer sang API design production.

9.4. Exit Ticket

40. Một điểm trong request flow trước đây bạn “nghĩ Spring tự làm” nhưng nay có thể giải thích rõ hơn?
41. Một API design anti-pattern bạn sẽ kiểm tra ngay trong group project?
42. Một claim từ AI bạn đã kiểm chứng hoặc bác bỏ bằng source/log/docs là gì?

PHỤ LỤC A – 135 PHÚT GỢI Ý CHO GIẢNG VIÊN

Thời gian	Hoạt động
0–15	Retrieval Slot 17 + baseline demo_slot18
15–35	REST Controller mental model + mappings/binding
35–55	ResponseEntity/status + CRUD contract
55–75	Live GET/POST trace Controller → DB
75–95	PUT/DELETE/search + relationship/JSON boundary
95–110	Break-It / diagnose 400/404/415/500
110–122	Production transfer: e-commerce API design
122–130	AI Self-Discovery Prompt 1 kickoff
130–135	Human Gate + Exit Ticket

PHỤ LỤC B – REQUEST-TO-DATABASE TRACE WORKSHEET

Step	Class/File	Method	Input	Output	Evidence
1					
2					
3					
4					

Step	Class/File	Method	Input	Output	Evidence
5					
6					
7					
8					

PHỤ LỤC C – API DESIGN REVIEW WORKSHEET

Question	Decision/Answer	Evidence/Reason
Resource noun rõ chưa?		
HTTP method đúng semantics?		
Path variable/query param/body dùng đúng vai trò?		
Success + error status code?		
DTO/Entity exposure decision?		
Validation rules?		
Pagination/filter/sort?		
Authorization boundary?		
Idempotency/concurrency risk?		
Versioning/deprecation need?		
Observability/audit need?		
Test strategy?		

PHỤ LỤC D – HUMAN VERIFICATION GATE

43. Mở DemoSlot18Application và giải thích từ main() đến khi server sẵn sàng.
44. Mở OrchidController và chỉ ra base path, GET list, GET detail, POST, PUT, DELETE.
45. Chọn một request và đọc method chain Controller → Service → Repository.
46. Chỉ ra chỗ Spring inject Service vào Controller và Repository vào Service.
47. Giải thích repository interface không có implementation class do sinh viên viết nhưng vẫn chạy vì sao.
48. Gửi GET missing id và giải thích vì sao phải là 404 thay vì 200 + null.
49. Gửi POST và chỉ ra JSON được bind vào object ở boundary nào.
50. Sau POST, chỉ ra generated ID và row/FK trong SQL Server.
51. Nếu gửi Content-Type text/plain, dự đoán response và kiểm chứng.
52. Nếu URL đúng nhưng method POST/GET sai, phân biệt 404 và 405.
53. Giải thích vì sao Service phù hợp làm transaction boundary.
54. Giải thích một nguy cơ khi trả JPA entity có bidirectional relationship.
55. Đưa ra một DTO cho Orchid response và giải thích field nào không nên expose.
56. Giải thích PUT và PATCH khác nhau theo contract.
57. Thiết kế search Product với filter/category/page/sort cho e-commerce.
58. Nêu một API cần idempotency và lý do.
59. Nêu một workflow không nên đặt trong một DB transaction duy nhất.
60. Chỉ ra một anti-pattern trong code beginner và refactor direction.
61. Đưa ra một claim từ Prompt 3 và chỉ ra official source đã kiểm chứng.

62. Nếu AI mô tả method không tồn tại trong demo_slot18, bạn xử lý thế nào?

PHỤ LỤC E – EVIDENCE PACKAGE & GIT CHECKPOINTS

ID	Evidence
E1	Baseline application + SQL Server connected
E2	Endpoint mapping/API contract
E3	GET list/detail evidence
E4	POST 201 + DB row/FK
E5	PUT before/after
E6	DELETE + post-condition
E7	Negative case 400/404/415
E8	Execution trace worksheet
E9	Break-It failure + fix
E10	Postman collection / Swagger
E11	AI Prompt 1 response + verification log
E12	E-commerce API design worksheet
E13	Human Gate notes
E14	README / Git history

Suggested checkpoints

V0 baseline-slot17
 V1 rest-controller-skeleton
 V2 get-endpoints
 V3 post-create
 V4 put-delete
 V5 search-negative-cases
 V6 relationship-json-verified
 V7 execution-trace
 V8 break-fix
 V9 lab04-complete
 V10 production-transfer-ai-verified

PHỤ LỤC F – QUICK REFERENCE: SPRING MVC REST

Mục tiêu	Annotation/API	Nhớ
REST controller	@RestController	HTTP boundary + body serialization
Base path	@RequestMapping	Class/method level criteria
Read	@GetMapping	GET semantics
Create	@PostMapping	Thường 201
Update	@PutMapping/@PatchMapping	Full vs partial theo contract
Delete	@DeleteMapping	Thường 204
Resource identity	@PathVariable	Path segment
Filter/options	@RequestParam	Query string

Mục tiêu	Annotation/API	Nhớ
JSON input	@RequestBody	Deserialize body
Validation	@Valid + constraints	Reject invalid request before business/persistence
Response control	ResponseEntity	Status + headers + body
Global errors	@RestControllerAdvice	Consistent contract
Controller test	@WebMvcTest + MockMvc	No live server
JPA slice	@DataJpaTest	Repository/mapping
Full context	@SpringBootTest	Integration-oriented

PHỤ LỤC G – TECHNOLOGY UPDATE 2026 & MIGRATION AWARENESS

Course baseline vs update: Giữ Lab/assessment trên JDK 21 + Spring Boot 3.x để ổn định. Phần này chỉ giúp sinh viên không “đóng băng” kiến thức; mọi migration cần project riêng và test đầy đủ.

Area	Course baseline	Current radar (06/10/2026)	Learning action
Spring Boot	3.x track	Official docs list 4.1.1, 4.0.8, 3.5.16... as stable lines.	Hiểu major migration; không tự nâng lab giữa kỳ.
Spring Framework	6.x with Boot 3	7.0.x stable generation under Boot 4.	Theo dõi MVC/API/testing changes.
Java/Jakarta	JDK 21; Jakarta packages	Framework 7 aligns with Jakarta EE 11; Boot 4 retains Java 17 baseline and supports newer Java.	Giữ Java 21 course; đọc migration notes.
API Versioning	Design strategy thủ công/course concepts	Spring Framework 7 has built-in MVC API versioning support.	So sánh framework support với path/header strategy cũ.
REST clients	RestTemplate/RestClient/WebClient awareness	Framework 7 deprecates RestTemplate in favor of RestClient; HTTP Service Clients supported.	Ưu tiên học RestClient/HTTP interfaces cho code mới khi stack phù hợp.
JSON	Jackson 2 in many Boot 3 projects	Framework 7 generation supports Jackson 3 direction.	Không hardcode assumptions; check Boot BOM/migration guide.
Testing	JUnit 5, MockMvc, Mockito	Framework 7 generation includes newer testing APIs/ecosystem.	Giữ test boundary mental model; update annotation/tool by version.
Observability	Logs + Actuator awareness	Boot 4.1 highlights OpenTelemetry/observability updates.	Học trace/metrics correlation sau core REST/JPA.

Official references (accessed 2026-10-06):

- Spring Boot documentation – stable versions: <https://docs.spring.io/spring-boot/>
- Spring Boot project page: <https://spring.io/projects/spring-boot>
- Spring Boot 4.0.0 release: <https://spring.io/blog/2025/11/20/spring-boot-4-0-0-available-now/>
- Spring Boot 4.1.0 release: <https://spring.io/blog/2026/06/10/spring-boot-4/>
- Spring Framework 7.0 GA: <https://spring.io/blog/2025/11/13/spring-framework-7-0-general-availability/>
- Spring MVC API Versioning: <https://docs.spring.io/spring-framework/reference/web/webmvc/mvc-config/api-version.html>
- Spring REST Clients: <https://docs.spring.io/spring-framework/reference/integration/rest-clients.html>
- Spring MVC ResponseEntity: <https://docs.spring.io/spring-framework/reference/web/webmvc/mvc-controller/ann-methods/responseentity.html>

PHỤ LỤC H – E-COMMERCE API DESIGN PACK

H.1. Resource map

Conceptual resource relationships

```

Customer
-> Cart -> CartItem -> Product
-> Order -> OrderItem -> Product snapshot
    |
    +-> Payment
    +-> Shipment
Product -> Category
Product -> Inventory / Reservation

```

H.2. Checkout workflow – không nhầm với CRUD đơn giản

Production-oriented conceptual flow

```

POST /api/v1/orders
-> OrderController.createOrder()
-> CheckoutService.checkout()
    -> CartService.validateCart()
    -> PricingService.calculateSnapshot()
    -> InventoryService.reserve()
    -> OrderRepository.save()
    -> Payment boundary (sync/async depending architecture)
    -> event/outbox/notification boundary
<- OrderResponse

```

Architecture caution: Ví dụ trên dùng để học boundary/trade-off. Không suy luận rằng mọi e-commerce phải dùng microservices, saga hay event-driven từ ngày đầu. Bắt đầu bằng modular boundaries rõ và đo nhu cầu thực tế.

H.3. API review questions cho Product/Order

- Resource identity ổn định chưa? Có dùng business key nào cần unique constraint?
- List endpoint có pagination/filter/sort và limit tối đa chưa?
- Create order/payment có duplicate-submit risk không?
- Error contract có đủ cho React client xử lý field/global errors không?
- Authorization kiểm ở policy/service boundary nào?
- Transaction boundary có kéo dài qua external network call không?
- Logs/traces có correlation id để theo workflow không?
- API version/deprecation có kế hoạch migration client không?

PHỤ LỤC I – INSTRUCTOR / PEER QUESTION BANK

Foundation

- DispatcherServlet làm gì trước khi controller method được gọi?
- Tại sao request mapping là một phần của contract?
- ResponseEntity khác return object trực tiếp ở khả năng nào?
- Jackson tham gia lúc request và response như thế nào?

Layering & JPA

- Nếu Controller gọi JpaRepository trực tiếp, điều gì vẫn chạy được và điều gì xấu đi?
- Khi nào @Transactional(readOnly=true) hữu ích?
- Repository proxy được tạo từ interface như thế nào về mặt khái niệm?
- Tại sao entity graph không nên mặc định trở thành API graph?

HTTP & production

- POST 201 vs 200: khác biệt nào quan trọng cho client?

- 400 và 404 khác nhau ở lỗi input/resource như thế nào?
- PUT vs PATCH: contract nào dễ bị hiểu sai?
- Tại sao payment/create-order cần quan tâm idempotency?
- Khi nào API versioning cần thiết và khi nào chỉ làm URL xấu hơn?
- Bạn sẽ đo N+1/latency ở list endpoint bằng evidence nào?

AI verification

- AI nói startup gọi method X nhưng source/stack trace không có: bạn làm gì?
- AI đề xuất một feature “mới nhất”: bạn cần nguồn nào trước khi tin?
- Làm sao phân biệt conceptual framework flow với exact internal method chain theo version?
- Một prompt tốt phải buộc AI đưa evidence/assumption như thế nào?

TÀI LIỆU THAM KHẢO VÀ GHI CHÚ PHẠM VI

- SBA301 Syllabus – CLO4 và course sequence Spring Boot/JPA.
- Chapter 13 – Connecting RESTful Web Service to JPA: objectives, project setup, domain model, data access, service, REST Controller, CRUD, additional considerations.
- Lab 04 – Building Spring Boot RESTful Web Services and Spring Data JPA: Orchid Management + SQL Server.
- SBA301 Slot 16 – Testing/MockMvc: testing boundary, Break-It, Human Verification, AI verification pattern.
- SBA301 Slot 17 – JPA/Entity/Repository/Service/Relationships: persistence mental model, relationship mapping, transaction and PA02 readiness.
- Spring official documentation/release notes listed in Appendix G for current technology awareness.

Scope lock: Slot 18 tập trung REST Controller + REST↔JPA integration. Security chuyên sâu, distributed systems, event-driven architecture và full production hardening chỉ dùng để định hướng/transfer, không thay core learning outcomes.